

Big data e ciência de dados

aula 12

Análises supervisionadas – modelos de classificação – Árvores de decisão

Luisa Mariele Strauss

Provost & Fawcett (2016)
Notas de aula Felipe Moraes e documentação python

```
mirror_mod = modifier_ob.  
    mirror object to mirror  
    mirror_mod.mirror_object  
    operation == "MIRROR_X":  
        mirror_mod.use_x = True  
        mirror_mod.use_y = False  
        mirror_mod.use_z = False  
    operation == "MIRROR_Y":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = True  
        mirror_mod.use_z = False  
    operation == "MIRROR_Z":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = False  
        mirror_mod.use_z = True
```

```
selection at the end -add  
mirror_ob.select= 1  
mirror_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier  
mirror_ob.select = 0  
= bpy.context.selected_obj  
data.objects[one.name].se  
print("please select exactly
```

```
--- OPERATOR CLASSES ---
```

```
types.Operator):  
    X mirror to the selected  
    object.mirror_mirror_x"  
    mirror X"
```

```
context):  
context.active_object is not
```

Regressão Logística e Árvores de decisão

- São técnicas para problemas de classificação, em que as variáveis alvo são **categóricas**
- Exemplos:
 - Bom/mau pagador
 - Aprovado/Reprovado
 - Churn (Cancelou/Não cancelou)

```
mirror_mod = modifier_ob.  
    mirror object to mirror  
    mirror_mod.mirror_object  
    operation == "MIRROR_X":  
        mirror_mod.use_x = True  
        mirror_mod.use_y = False  
        mirror_mod.use_z = False  
    operation == "MIRROR_Y":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = True  
        mirror_mod.use_z = False  
    operation == "MIRROR_Z":  
        mirror_mod.use_x = False  
        mirror_mod.use_y = False  
        mirror_mod.use_z = True
```

```
selection at the end -add  
mirror_ob.select= 1  
mirror_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier  
mirror_ob.select = 0  
= bpy.context.selected_obj  
data.objects[one.name].se  
print("please select exactly
```

```
--- OPERATOR CLASSES ---
```

```
types.Operator):  
    X mirror to the selected  
    object.mirror_mirror_x"  
    mirror X"
```

```
context):  
context.active_object is not
```

Regressão Logística

- Equação da função logística:

$$p(x) = \frac{1}{1+e^{-f(x)}}, \text{ onde}$$

$p(x)$: probabilidade de X (a variável dependente que desejamos prever)

e: número de Euler (2,71)

$f(x)$: coeficientes das variáveis

independentes, definidas como $\beta_0 + \beta_1 * x_1 + \beta_2 * x_2 + \dots + \beta_n * x_n$


```

mirror_mod = modifier_ob.
    mirror object to mirror_
    mirror_mod.mirror_object
    operation == "MIRROR_X":
    mirror_mod.use_x = True
    mirror_mod.use_y = False
    mirror_mod.use_z = False
    operation == "MIRROR_Y":
    mirror_mod.use_x = False
    mirror_mod.use_y = True
    mirror_mod.use_z = False
    operation == "MIRROR_Z":
    mirror_mod.use_x = False
    mirror_mod.use_y = False
    mirror_mod.use_z = True

```

```

    selection at the end -add
    mirror_ob.select= 1
    mirror_ob.select=1
    context.scene.objects.active
    ("Selected" + str(modifier_
    mirror_ob.select = 0
    = bpy.context.selected_ob
    data.objects[one.name].se
    print("please select exactly

```

----- OPERATOR CLASSES -----

```

types.Operator):
    X mirror to the selected
    object.mirror_mirror_x"
    mirror X"

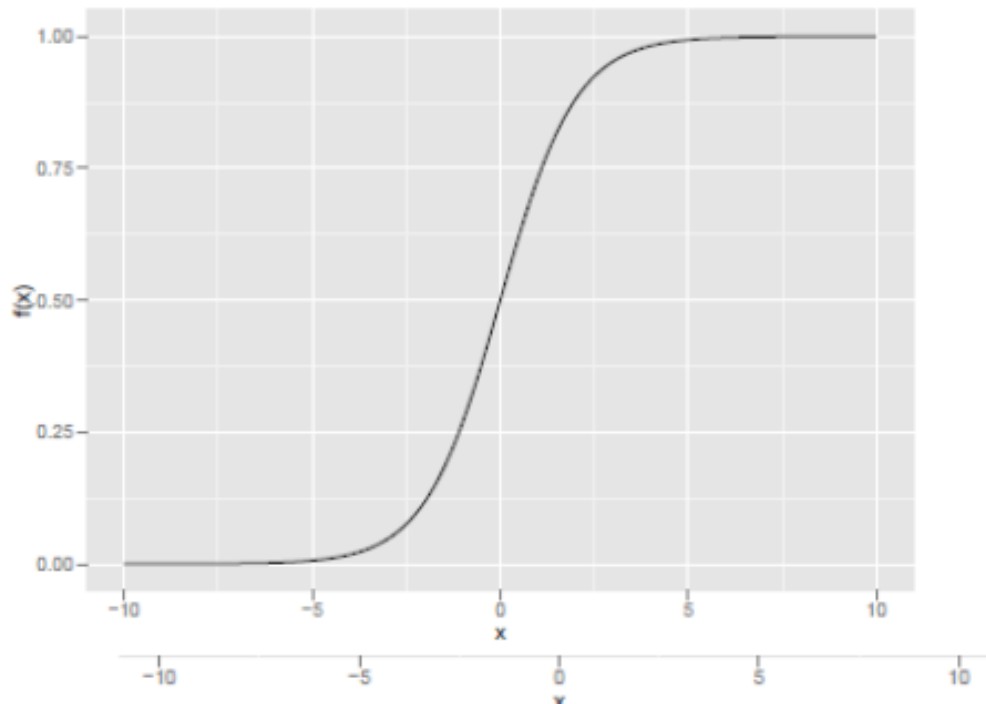
```

```

context):
    context.active_object is not

```

Regressão Logística



Exemplo regressão logística: previsão de cancelamento (churn)

- Base: tele_customer_data.csv
- Contém dados de clientes de uma telecom, com alguns valores de perfil e outros de consumo, e a coluna *churn* indica clientes que cancelaram o serviço no último mês



Equação resultante

- $p(x) = \frac{1}{1+e^{-f(x)}}$

- $f(x) = \beta_0 + \beta_1 * x_1 + \beta_2 * x_2 + \dots + \beta_n * x_n$

- $P(x)$: probabilidade de churn

- Coeficientes ($\beta_1, \beta_2, \dots, \beta_n$): $[-0.4 \ -0.12 \ -0.05 \ 0.26 \ -0.21 \ -0.23 \ 0.22]$

- Intercepto (β_0): $[-0.82]$

Como os resultados são apresentados na regressão logística?

- Como percentuais de probabilidade e a predição (baseada no **limiar de decisão**)

[0.75407593 0.24592407]: 0 → não churn

[0.76054283 0.23945717] : 0 → não churn

[0.76905403 0.23094597] : 0 → não churn

[0.82350259 0.17649741] : 0 → não churn

[0.69910862 0.30089138] : 0 → não churn

[0.65466457 0.34533543] : 0 → não churn

[0.54583559 0.45416441] : 0 → não churn

[0.8039427 0.1960573] : 0 → não churn

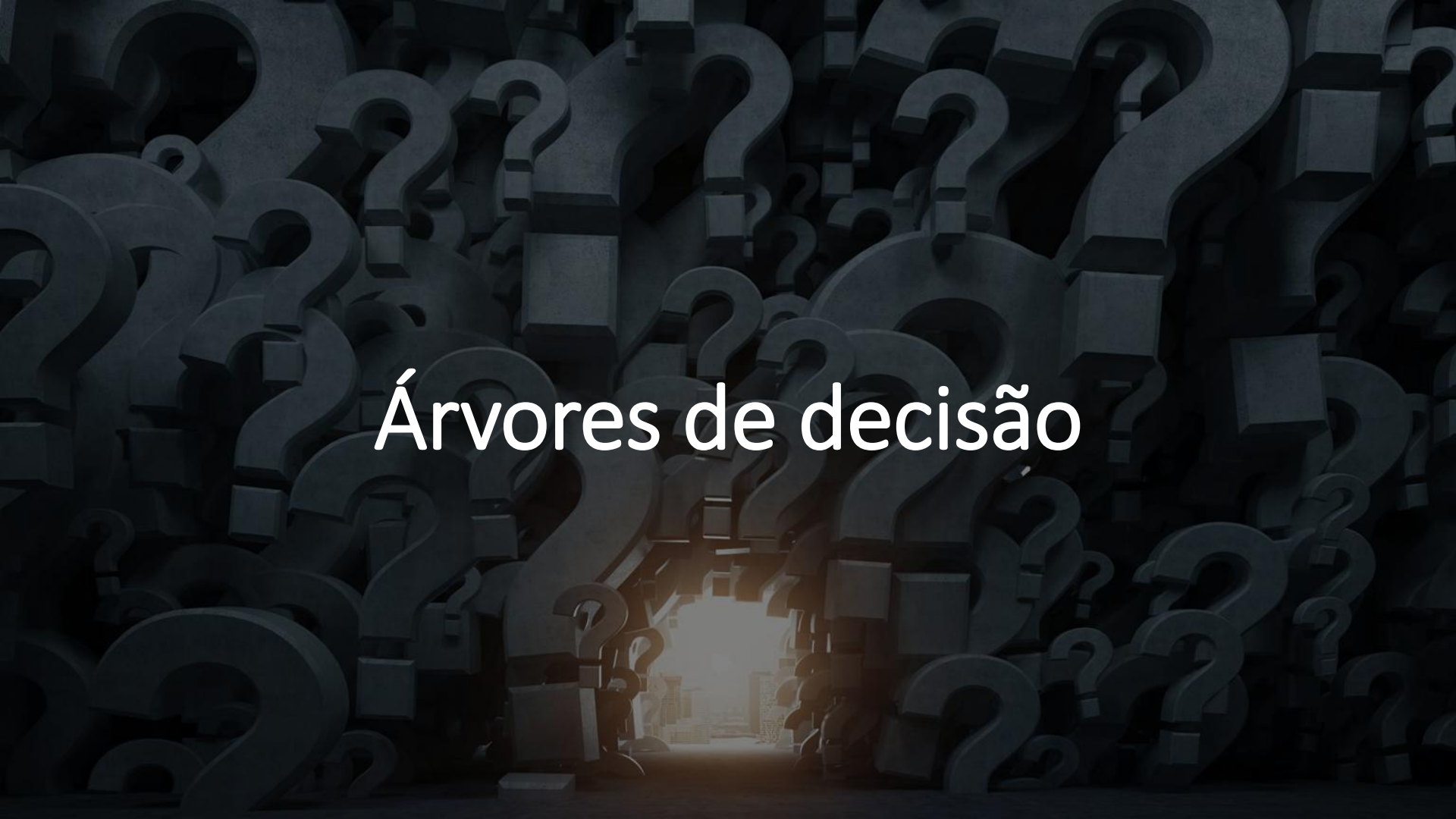
[0.45769605 0.54230395] : 1 → churn

[0.74822579 0.25177421] : 0 → não churn



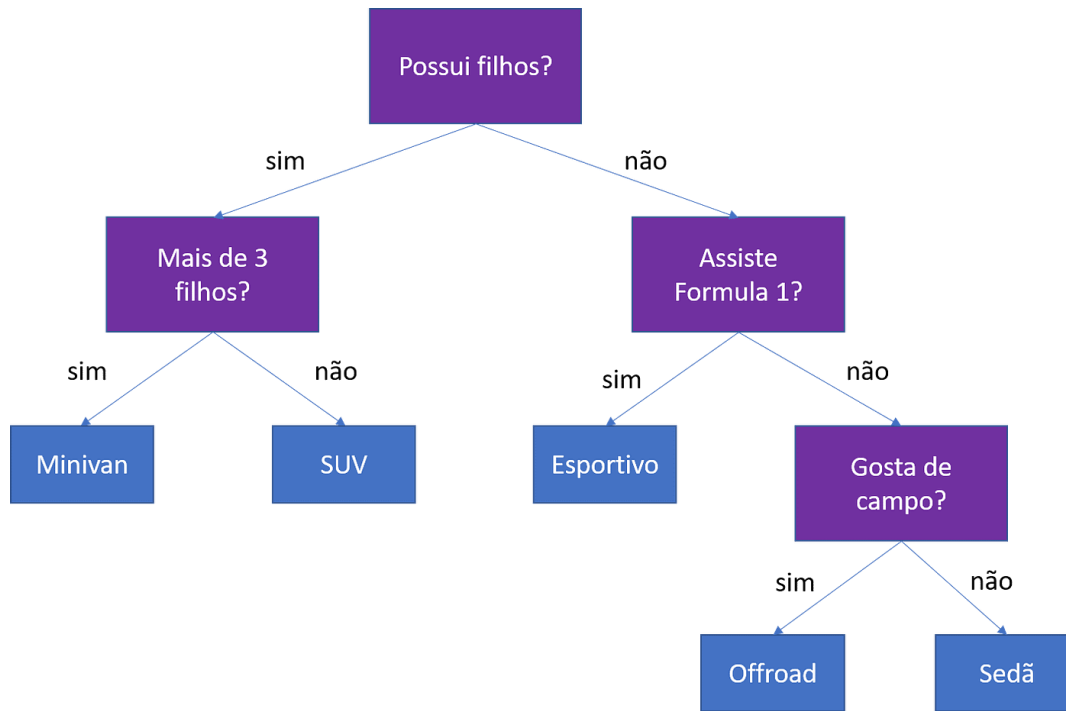
Como isso
poderia se
transformar em
um sistema de
apoio à decisão?



The background of the slide is a dark, monochromatic field filled with numerous 3D question marks. These question marks are rendered in a dark grey or black color with a matte finish. They are scattered across the entire frame, creating a sense of depth and complexity. In the lower center of the image, there is a bright, glowing light source that creates a strong lens flare and illuminates the surrounding question marks, casting soft shadows and highlighting their three-dimensional form. The overall composition suggests a state of uncertainty or a path leading through a maze of questions.

Árvores de decisão

Árvore de decisão para compra de carro

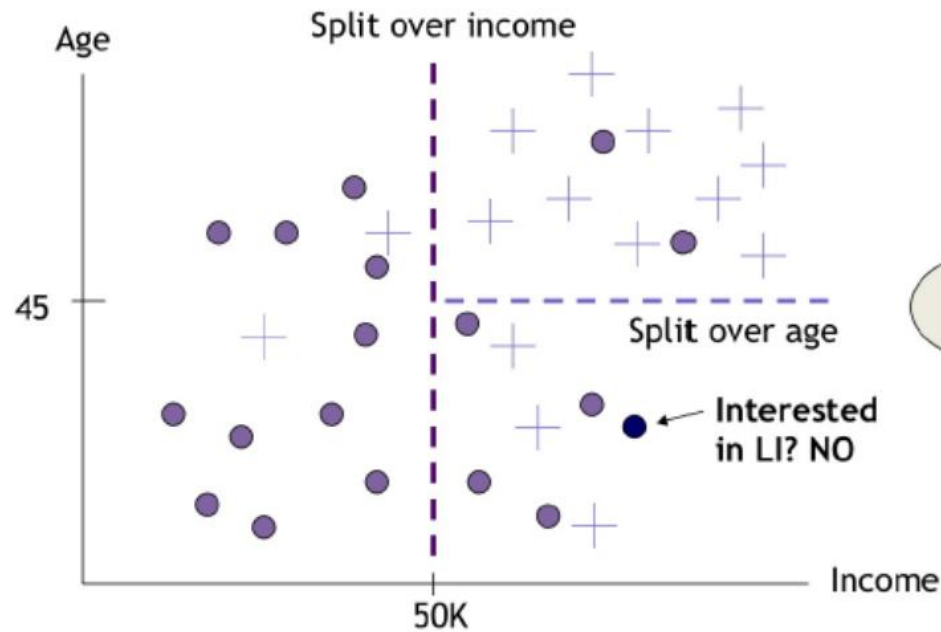


Árvores de classificação

- É um tipo de modelo estruturado de árvore de decisão que é criado a partir dos dados*.
- São populares por conta da facilidade de entendimento, fáceis de usar e fazer previsões.

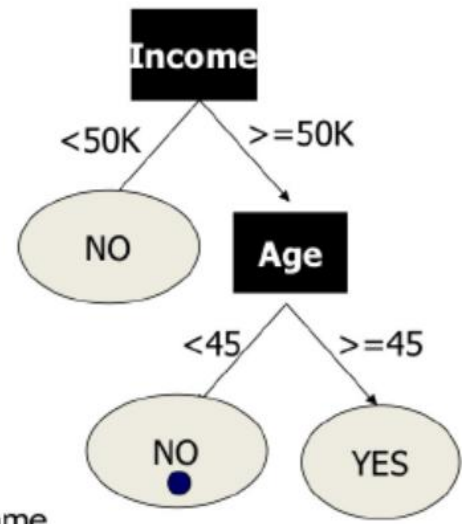
*Qual a diferença entre um modelo em ML e baseado em regras?





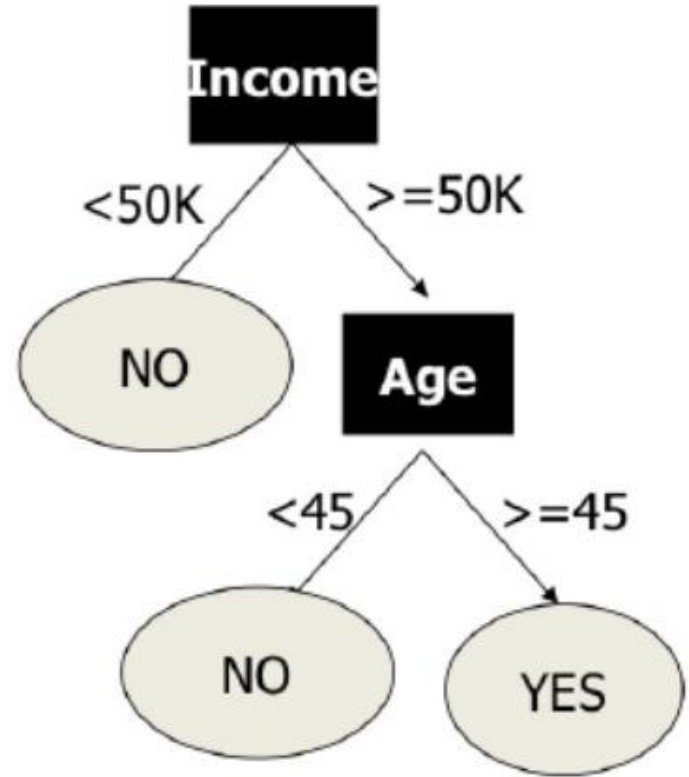
- Did not buy life insurance (response = no)
- + Bought life insurance (response = yes)

Classification tree



Características

- As árvores criam uma segmentação dos dados – baseado em regras
- Cada **nodo** contém um teste de um atributo
- Cada caminho em algum momento termina numa **folha**
- Cada folha corresponde a um segmento, e os atributos ao longo do caminho dão as características
- Cada folha contém um valor para a variável alvo



Animação de como funciona o algoritmo



https://youtu.be/ZVR2Way4nwQ?si=vgnQLQGJMBD1F_1E

Voltando ao exemplo anterior, mas usando árvore de classificação: previsão de cancelamento (churn)

- Base: tele_customer_data.csv
- Contém dados de clientes de uma telecom, com alguns valores de perfil e outros de consumo, e a coluna *churn* indica clientes que cancelaram o serviço no último mês



No python

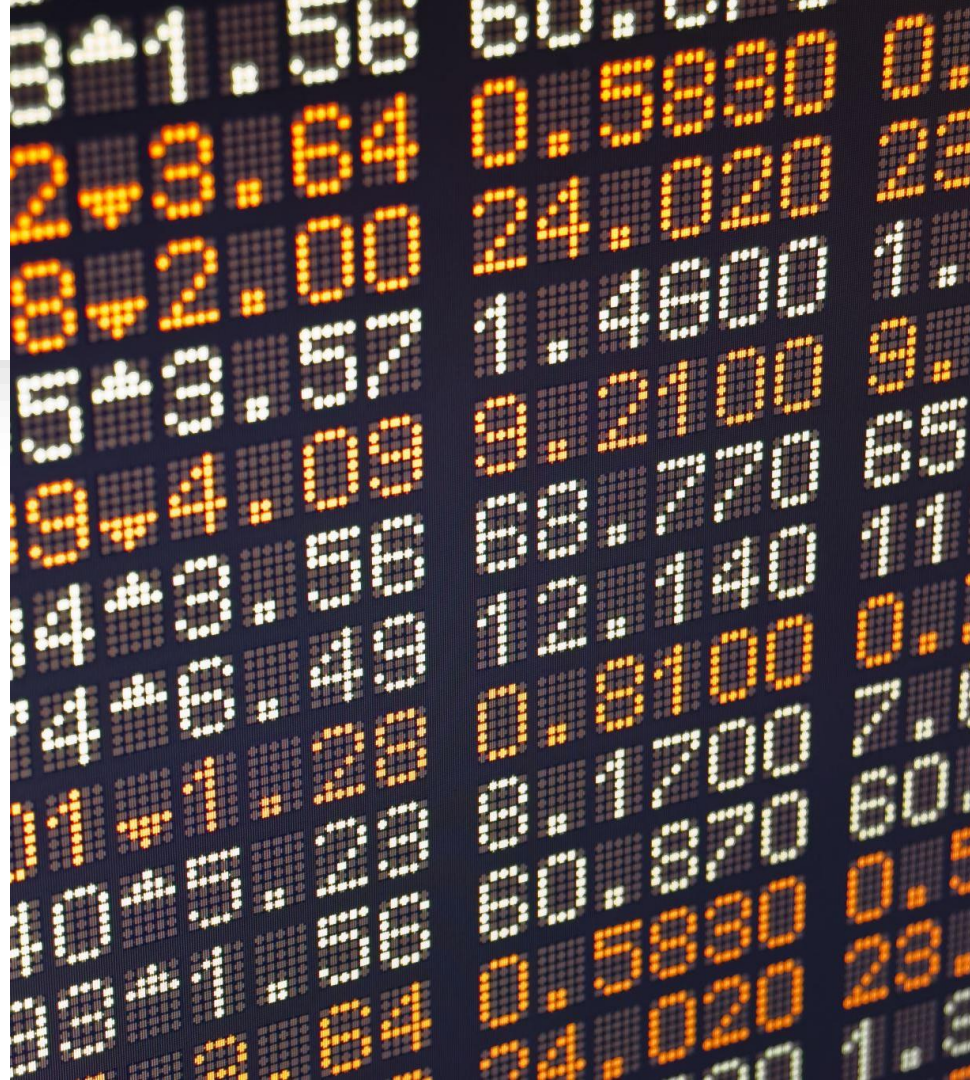
```
# Instalar as bibliotecas a seguir para  
# visualizar a árvore de decisão  
!pip install graphviz pydotplus imbalanced-learn  
  
from sklearn.tree import DecisionTreeClassifier  
# Biblioteca para rodar o algoritmo de árvore  
# de decisão  
from sklearn.tree import export_text,  
export_graphviz # Caso se deseje ver a árvore  
# de decisão em formato de texto  
import pydotplus # Para visualizar a árvore  
from IPython.display import Image
```

- Qual dos modelos é melhor para prever churn?



Avaliando modelos de classificação

- Matriz de confusão
- Acurácia
- Taxa de erro
- Precisão
- Recall



Matriz de confusão

		Previsto	
		Positivo	Negativo
Valor real	Positivo	Verdadeiro positivo	Falso negativo
	Negativo	Falso positivo	Verdadeiro negativo

Cross-validation

- Além de separar os dados em teste e treinamento e avaliar os resultados com os indicadores gerados a partir da matriz de confusão, é recomendado que testemos também os **parâmetros** usados no algoritmo
- Exemplos:
 - k-means, K-medoids e k- prototypes: gerar vários centróides aleatórios e testar para vários números de clusters (k)
 - Regressão linear múltipla: remover e inserir variáveis e testar a melhor equação resultante
 - Árvore de decisão: testar com diferentes profundidades e número de indivíduos por folha



Método GridSearchCV

- Biblioteca scikit-learn

```
from sklearn.model_selection import GridSearchCV
```

- Definimos os hiperparâmetros que serão testados
 - São as definições prévias do algoritmo
- O método então elabora uma grade com todas as combinações
- Testagem dos resultados nos dados de teste
- Definir qual pontuação será considerada: acurácia, recall, precisão ou f1-score (modelos de classificação); R^2 ou R^2 ajustado em regressão; método do cotovelo para clusterização etc.

